

We call `_create_alien()`, and pass it the `y`-value as well as its `x`-position ❹. We'll modify `_create_alien()` in a moment.

Notice the indentation of the last two lines of code ❺. They're inside the outer `while` loop, but outside the inner `while` loop. This block runs after the inner loop is finished; it runs once after each row is created. After each row has been added, we reset the value of `current_x` so the first alien in the next row will be placed at the same position as the first alien in the previous rows. Then we add two alien heights to the current value of `current_y`, so the next row will be placed further down the screen. Indentation is really important here; if you don't see the correct fleet when you run *alien_invasion.py* at the end of this section, check the indentation of all the lines in these nested loops.

We need to modify `_create_alien()` to set the vertical position of the alien correctly:

```
def _create_alien(self, x_position, y_position):
    """Create an alien and place it in the fleet."""
    new_alien = Alien(self)
    new_alien.x = x_position
    new_alien.rect.x = x_position
    new_alien.rect.y = y_position
    self.aliens.add(new_alien)
```

We modify the definition of the method to accept the `y`-value for the new alien, and we set the vertical position of the `rect` in the body of the method.

When you run the game now, you should see a full fleet of aliens, as shown in Figure 13-4.

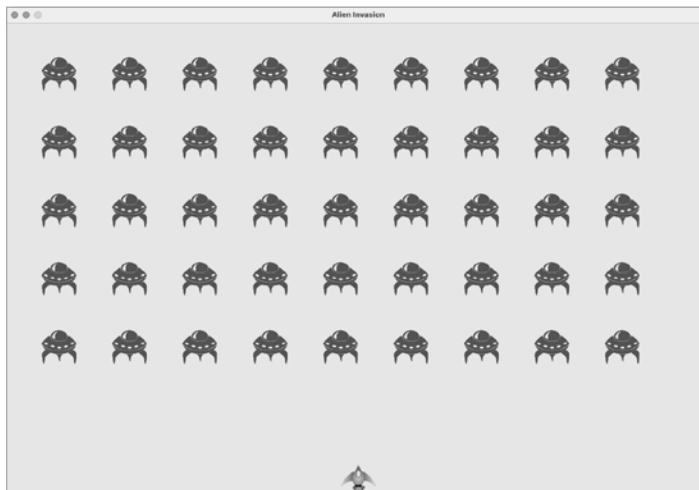


Figure 13-4: The full fleet appears.

In the next section, we'll make the fleet move!